Computer Graphics Assignment IV

Geometry

This assignment is 9% of the total mark, with up to 2% extra credit.

Student ID: _____

Student Name: _____

This assignment represents my own work in accordance with University regulations.

Signature: _____

Overview

Congratulations! In the last assignment, you built a rasterization pipeline. This time, you will dig deeply into geometry. This project has 2 parts and 6 tasks, worth a total of 110 points: 90 regular points plus 20 extra-credit points for additional work. The tasks are as follows:

- Part I: Bezier Curves and Surfaces (25 points)
  - Task 1: Bezier curves with 1D de Casteljau subdivision (10 points)
  - Task 2: Bezier surfaces with separable 1D de Casteljau (15 points)
- Part II: Triangle Meshes and the Half-Edge Data Structure (65 points)
  - Task 3: Area-weighted vertex normals (10 points)
  - Task 4: Edge flip (15 points)
  - Task 5: Edge split (15 points + 6 extra)
  - Task 6: Loop subdivision for mesh upsampling (25 points + 7 + 7 extra)

Building and Running the Executable

From the `Assignment4Code` directory, build the project with CMake:

```
$ mkdir build && cd build
$ cmake .. && make
```

The general command for running the executable `meshedit` (from the `build` directory) is:

```
./meshedit <PATH_TO_FILE>
```

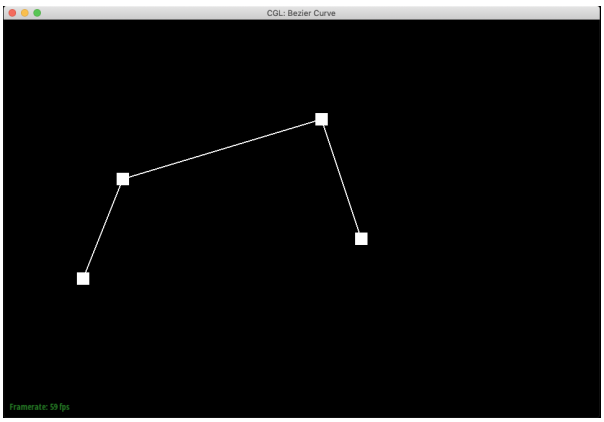
Note that `meshedit` expects different file formats for different parts of this assignment:

```
./meshedit ../bzc/curve1.bzc      // e.g., Part I, Task 1
./meshedit ../bez/teapot.bez      // e.g., Part I, Task 2
./meshedit ../dae/teapot.dae      // e.g., Part II
```

Using the Graphical User Interface (GUI)

The following details how to use the GUI provided by `meshedit`. You may want to skim through it on your first read-through, and read it in more detail once you have implemented parts of this assignment and are ready to use the GUI.

Task 1 (.bzc files) When you run `meshedit` with a Bezier curve file (`.bzc`) for Task 1, you will see the control points rendered on screen, as shown below.



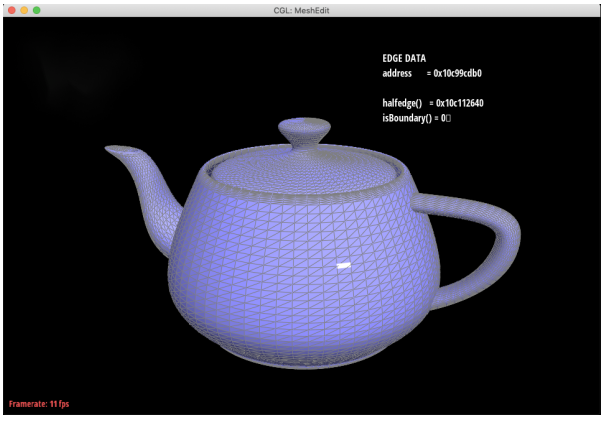
For this task only, the GUI is different. Below is the full specification of the keyboard controls.

Key	Action
E	Perform one `BezierCurve::evaluateStep(...)` call; cycle through the levels
C	Toggle whether or not the fully evaluated Bezier curve is drawn to the screen

To verify that your implementation is correct, you can repeatedly press E to cycle through each level of the de Casteljau subdivision, and press C to toggle the Bezier curve to check that it is generated correctly from the control points. You can also use your mouse to:

- click and drag the control points to move them, and see how your Bezier curve, along with all intermediate control points, changes accordingly;
- scroll to move the evaluated point along the Bezier curve and see how the intermediate control points move along with it. This essentially varies t between 0.0 and 1.0.

Task 2 onwards (.bez and .dae files) When you run `meshedit` for Task 2 and onwards, you will see a triangle mesh rendered on screen, as shown below.



As you hover the cursor around the screen, you will notice that mesh elements (half-edges, vertices, edges and faces) under the cursor are highlighted in purple or white, such as the edge

in the image above. You can click on one of these elements to select it, and the GUI will display some information about the element and its associated data. Below is the full specification of the keyboard controls.

Key	Action
Q	Toggle using vertex normals (Task 3)
F	Flip the selected edge (Task 4)
S	Split the selected edge (Task 5)
L	Upsample the current mesh (Task 6)
N	Select the next half-edge
T	Select the twin half-edge
W	Toggle wireframe
I	Toggle the information overlay
SPACE	Reset the camera to the default position
0–2	Switch the shader output mode

You can also use your mouse to:

- click and drag a vertex to change its position;
- click and drag the background, or right-click and drag anywhere, to rotate the camera;
- scroll to adjust the camera zoom.

You will implement area-weighted vertex normals Q, local edge flip F, local edge split S, and Loop subdivision L in Tasks 3, 4, 5 and 6, respectively. These four keys will do nothing until you have completed the corresponding task. Likewise, until Task 2 is implemented, loading a `.bez` file stops with the message "Error converting polygons to halfedge mesh"; this is expected.

Starter Code Structure

For Bezier curves and surfaces (Part I), you will fill in member functions of the `BezierCurve` and `BezierPatch` classes, defined in `bezierCurve.h` and `bezierPatch.h`.

For triangle meshes (Part II), you will fill in member functions of the `Vertex` and `HalfedgeMesh` classes, defined in `halfEdgeMesh.h`, and of the `MeshResampler` class, defined in `student_code.h`.

All the functions you need to modify have dummy definitions in `student_code.cpp`, and you will implement every task of this assignment in this file!

1 Part I: Bezier Curves and Surfaces (25 Points)

In computer graphics, Bezier curves and surfaces are frequently used to model smooth and infinitely scalable curves and surfaces, such as in Adobe Illustrator (a curve drawing program)

and in Blender (a surface modeling program).

A Bezier curve of degree n is defined by $(n+1)$ control points. It is a parametric curve based on a single parameter t, ranging between 0 and 1. Similarly, a Bezier surface of degree (n, m) is defined by $(n+1) \times (m+1)$ control points. It is a parametric surface based on two parameters u and v, both ranging between 0 and 1.

In Part I, you will use de Casteljau's algorithm to evaluate Bezier curves and surfaces for any given set of control points and parameters, such as the Bezier curve in Figure 1. In the image, white squares are the given control points, blue squares are the intermediate control points evaluated at the given parameter by de Casteljau's algorithm, and the red square is the final evaluated point, which lies on the Bezier curve.

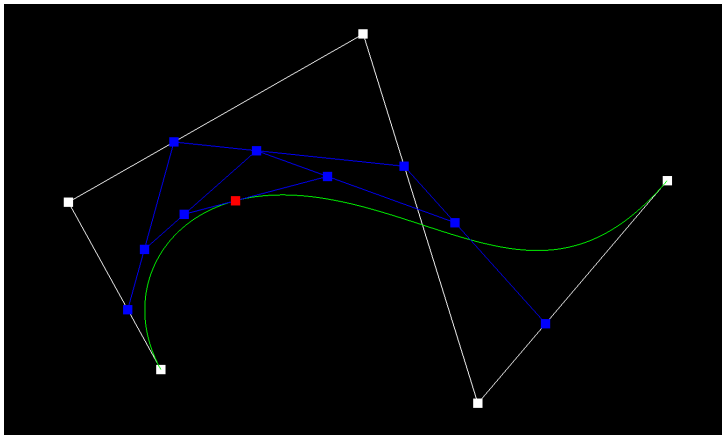


Figure 1: Bezier curve

1.1 Task 1: Bezier Curves with 1D de Casteljau Subdivision (10 Points)

In Task 1, you will implement Bezier curves. To get started, take a look at `bezierCurve.h` and examine the member variables defined in the class. Specifically, you will primarily be working with the following:

- `std::vector<Vector2D> controlPoints`: a `std::vector` of the original control points that define the Bezier curve, initialized from the input Bezier curve file (`.bzc`).
- `float t`: the parameter at which to evaluate the Bezier curve, ranging between 0 and 1.

Recall from lecture that de Casteljau's algorithm gives us the following recursive step. Given n (possibly intermediate) control points $p_1, \ldots, p_n$ and the parameter t, we can use linear interpolation to compute the $n-1$ intermediate control points at the parameter t in the next subdivision level, $p'_1, \ldots, p'_{n-1}$, where

$$p'_i = \text{lerp}(p_i, p_{i+1}, t) = (1-t)\, p_i + t\, p_{i+1}. \tag{1}$$

By applying this step recursively, we eventually arrive at a single final point, and this point in fact lies on the Bezier curve at the given parameter t.

You need to implement this recursive step in `BezierCurve::evaluateStep(...)` in `student_code.cpp`. This function takes as input a `std::vector` of 2D points and outputs a `std::vector` of the intermediate control points at the parameter t in the next subdivision level. Note that t is a member variable of the `BezierCurve` class, which you can access within the function. Each call to this function should perform only *one* step of the algorithm, i.e., one level of subdivision.

Implementation Notes `std::vector` is similar to Java's `ArrayList` class. Use the `push_back(...)` method to add elements to a `std::vector`; this is analogous to the `append(...)` method of `ArrayList`. See this page for more information on `push_back(...)`.

Sanity Check To check whether your implementation is likely correct, run `meshedit` with a Bezier curve file (`.bzc`) and view the generated Bezier curve on screen, using the Task 1 controls described above. For example:

```
./meshedit ../bzc/curve1.bzc
```

Here `bzc/curve1.bzc` is a cubic Bezier curve; the provided `bzc/curve2.bzc` is a degree-4 Bezier curve. Feel free to create your own `.bzc` files and explore other Bezier curves. (Format: the first line gives the number of control points, followed by one x y pair per control point, with coordinates between 0 and 1.)

Complete the given function:

```
BezierCurve::evaluateStep(...)
```

For Your Write-Up: Task 1

- Briefly explain de Casteljau's algorithm and how you implemented it to evaluate Bezier curves.
- Take a look at the provided `.bzc` files and create your own Bezier curve with 6 control points of your choosing. Use this Bezier curve for the screenshots below.
- Show screenshots of each step / level of the evaluation, from the original control points down to the final evaluated point. Press E to step through. Toggle C to show the completed Bezier curve as well.
- Show a screenshot of a slightly different Bezier curve, obtained by moving the original control points around and modifying the parameter t by mouse scrolling.

1.2 Task 2: Bezier Surfaces with Separable 1D de Casteljau (15 Points)

In Task 2, you will adapt what you have implemented for Bezier curves to Bezier surfaces. To get started, take a look at `bezierPatch.h` and examine the member variables defined in the class. Specifically, you will primarily be working with the following:

- `std::vector<std::vector<Vector3D>> controlPoints`: a 2D `std::vector` holding an $n \times n$ grid of original control points. `controlPoints` has size n, and each inner `std::vector`, i.e. `controlPoints[i]`, contains the n control points of the i-th row. For example, `controlPoints[0]` and `controlPoints[n-1]` each hold n control points, of the bottom row and the top row respectively.

Recall from lecture that the separable 1D de Casteljau algorithm works as follows. The inputs are (1) an $n \times n$ grid of original control points P_{ij}, where i and j are the row and column indices, and (2) the two parameters u and v.

We first consider each row of n control points, $P_{i0}, \ldots, P_{i(n-1)}$, as defining a Bezier curve parameterized by u. Just as in Task 1, we can use the recursive step to evaluate the final, single point P_i on this Bezier curve at the parameter u. We then consider the points P_i of all n rows as defining a Bezier curve parameterized by v. (These n points lie roughly along a column; the Bezier surface slides from Lecture 11 may help you visualize this.) We can again use the recursive step from Task 1 to evaluate the final, single point P on this Bezier curve at the parameter v. This point P in fact lies on the Bezier surface at the given parameters u and v.

You need to implement the following functions in `student_code.cpp`:

- `BezierPatch::evaluateStep(...)`: this recursive function works like `BezierCurve::evaluateStep(...)` from Task 1. It takes as input a `std::vector` of 3D points and a parameter t. It outputs a `std::vector` of the intermediate control points at the parameter t in the next subdivision level.
- `BezierPatch::evaluate1D(...)`: this function takes as input a `std::vector` of 3D points and a parameter t. It outputs directly the final, single point that lies on the Bezier curve at the parameter t; it does not output intermediate control points. You may want to call `BezierPatch::evaluateStep(...)` inside this function.
- `BezierPatch::evaluate(...)`: this function takes as input the parameters u and v and outputs the point that lies on the Bezier surface, defined by the $n \times n$ `controlPoints`, at the parameters u and v. Note that `controlPoints` is a member variable of the `BezierPatch` class, which you can access within this function.

Sanity Check To check whether your implementation is likely correct, run `meshedit` with a Bezier surface file (`.bez`) and view the generated Bezier surface on screen. For example:

```
./meshedit ../bez/teapot.bez
```

Complete or use the given functions:

```
BezierPatch::evaluateStep(...)
BezierPatch::evaluate1D(...)
BezierPatch::evaluate(...)
```

For Your Write-Up: Task 2

- Briefly explain how de Casteljau's algorithm extends to Bezier surfaces and how you implemented it to evaluate Bezier surfaces.
- Show a screenshot of `bez/teapot.bez` (not `.dae`) evaluated by your implementation.

2 Part II: Triangle Meshes and the Half-Edge Data Structure (65 Points)

Note

- In Part II, you will work extensively with the half-edge data structure, as well as the provided `HalfedgeMesh` class, which implements the data structure.
- Before diving into this part, we recommend that you review the half-edge data structure from Lecture 12 (Mesh and Geometry Processing).
- We also highly recommend that you carefully read the documentation at the beginning of `src/halfEdgeMesh.h`. It explains how to navigate meshes with the `HalfedgeMesh` class, with examples of iterating around faces and vertices and of handling boundary loops, which you will find very helpful for this part.

In Part I, you implemented Bezier surfaces, which are parametric surfaces defined by a 2D grid of control points. As discussed in lecture, we can also represent surfaces using triangle meshes. While Bezier surfaces are better at representing smooth surfaces than triangle meshes and require less memory, Bezier surfaces are much more difficult to render directly. (Can you reason why?) In fact, Bezier surfaces are often converted into triangle meshes before being rendered to the screen.

As a result, triangle meshes are often the preferred way to represent 3D geometric models in computer graphics. One way to store a triangle mesh is as a list of vertices and a list of triangles indexing the vertices (see Lecture 12). Unfortunately, such a simple data structure does not allow the efficient traversal of meshes required by many geometry processing tasks. For example, to find all triangles neighbouring a given triangle, we must iterate through the entire list of triangles, which can be prohibitively expensive if the mesh has millions of triangles.

To answer the neighbouring-triangle query and many others more efficiently, the half-edge data structure explicitly stores the connectivity information among mesh elements, i.e., vertices, edges and faces. Throughout this part, you will use the half-edge data structure to implement a few commonly used geometric operations, and we hope you will come to appreciate the simplicity and flexibility of this data structure.

2.1 Task 3: Area-Weighted Vertex Normals (10 Points)

In Task 3, you will implement area-weighted normal vectors at vertices. Recall from lecture that these vertex normals can be used for Phong shading, which provides better shading for smooth surfaces than flat shading. To get started, take a look at the `Vertex` class defined in `halfEdgeMesh.h`, along with the very helpful comments within the class. In short, a `Vertex` object encapsulates a single mesh vertex and has members such as the following (the list is not exhaustive):

- `Vector3D position`: the 3D coordinates of this vertex.
- `HalfedgeIter& halfedge()`: returns a reference to the half-edge rooted at this vertex.
- `HalfedgeCIter halfedge() const`: returns the same half-edge as above, except that it is `const` (i.e., constant) and cannot be modified.

To compute an area-weighted normal at a given vertex, use the half-edge data structure to iterate through the faces (triangles) incident to the vertex, i.e., the faces that have the given vertex as one of their vertices. For each such face, weight its normal by its area. Finally, normalize the sum of all the area-weighted normals.

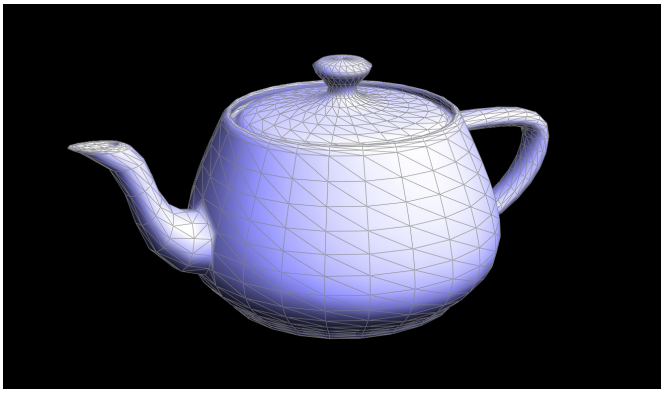
You need to implement this in `Vertex::normal()` in `student_code.cpp`. This function takes no input and outputs the area-weighted vertex normal. Since `normal()` is a member function of the `Vertex` class, you have access to the class members, such as `Vector3D position`, within the function. To iterate through the faces incident to a vertex, follow the example of iterating around a vertex in the documentation at the top of `halfEdgeMesh.h`. (Hint: a face in the half-edge data structure only points to its associated half-edge. To compute the area and normal of a face, what you really need are its three vertices rather than the face itself. A normal of a face is a vector perpendicular to the surface; the cross product of two vectors along the face returns a third vector perpendicular to both.)

Implementation Notes For Task 3 only, you are implementing a `const` member function, which requires that no code within the function modifies any member variables. This just means that you should use `HalfedgeCIter` and not `HalfedgeIter` in `Vertex::normal()`. (`HalfedgeIter` is an iterator into the mesh's list of half-edges, `HalfedgeCIter` is its `const` version, and `Halfedge*` is a plain pointer; the comments in `halfEdgeMesh.h` explain the differences.) For vector operations such as `cross(...)`, `dot(...)`, `norm()` and `unit()`, see the `Vector3D` class in `CGL/include/CGL/vector3D.h`.

Sanity Check To check whether your implementation is likely correct, run `meshedit` with a COLLADA mesh file (`.dae`). Once the model is loaded, press Q to toggle the area-weighted normals, which calls your `Vertex::normal()`; the shading of the model should become smooth instead of flat. For example:

```
./meshedit ../dae/teapot.dae
```

After you press Q, the shading of the teapot should look smooth, like the image below. If your shading differs from the image, your implementation of `Vertex::normal()` is likely incorrect. One common mistake is that the computed normals point towards the interior of the model, as opposed to the exterior as required in computer graphics. Oppositely oriented normals lead to very dark shading, which can be fixed by reversing the normals.



Complete or use the given function:

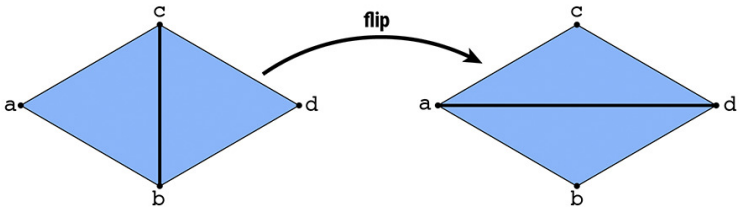
```
Vertex::normal()
```

For Your Write-Up: Task 3

- Briefly explain how you implemented the area-weighted vertex normals.
- Show screenshots of `dae/teapot.dae` (not `.bez`) comparing the teapot's shading with and without vertex normals. Use Q to toggle between the default flat shading and Phong shading.

2.2 Task 4: Edge Flip (15 Points)

In Task 4, you will implement a local remeshing operation on an edge, called a *flip*. Given a pair of triangles (a, b, c) and (c, b, d), a flip operation on their shared edge (b, c) converts the original pair of triangles into a new pair (a, d, c) and (a, b, d), as shown below:



You need to implement this in `HalfedgeMesh::flipEdge(...)` in `student_code.cpp`. This function takes as input an `EdgeIter` to the edge to be flipped, e.g. edge (b, c) above, and outputs an `EdgeIter` to the now flipped edge, e.g. edge (a, d) above. A good way to proceed:

- Draw a simple mesh, such as the pair of triangles (a, b, c) and (c, b, d) above, and write down a list of all its elements, i.e., half-edges, vertices, edges and faces.
- Draw the mesh after the remeshing operation and again write down a list of all elements in the modified mesh.
- If the remeshing operation adds new elements, create them. An edge flip does not add new elements, but an edge split in Task 5 does.
- For every element in the modified mesh, set all of its pointers to the correct element, *even if the element being pointed to has not changed*:
  - For each vertex, edge and face, set its `halfedge` pointer.
  - For each half-edge, set its `next`, `twin`, `vertex`, `edge` and `face` pointers. You can use `Halfedge::setNeighbors(...)` to set all the pointers of a half-edge at once.
  - We recommend setting all pointers of all elements in the modified mesh, not just the ones that changed, because it is very easy to miss a pointer and get errors that are very difficult to debug. Once you are sure that your implementation works, you can remove the unnecessary assignments if you wish.

Your implementation of `HalfedgeMesh::flipEdge(...)` should:

- Never flip a boundary edge. The function should simply return immediately if either neighbouring face of the edge is a boundary loop. Every mesh element has a useful `isBoundary()` function that returns true if the element is on the boundary.
- Perform only a constant amount of work. The cost of flipping a single edge should not be proportional to the mesh size.
- Not add or delete any mesh elements. There should be exactly the same number of elements before and after the edge flip; you only need to reassign pointers.

Implementation Notes Given any mesh element, you can use the debugging function `check_for(...)`, declared in `halfEdgeMesh.h`, to check which other elements in the mesh point to it. For example, you can use it to confirm that an element is pointed to by the right number of other elements.

Sanity Check After loading a model from a `.dae` file, click to select an edge and press F to flip it (see the controls for Task 2 onwards above). Sometimes your implementation may seem to work when you flip an edge only once, so flip an edge a few more times to be more certain that your code really works as expected. Task 6 will require a working implementation of edge flips. An edge flip should never create holes in your mesh. Note that you may reach a degenerate case where one triangle looks much darker than the others; this is fine.

Complete or use the given function:

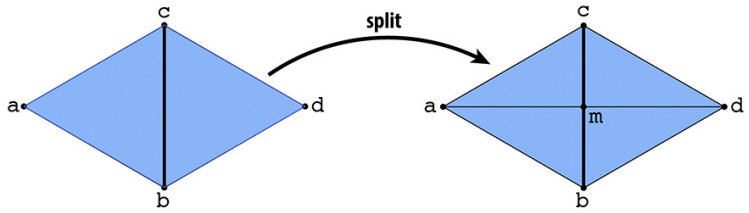
```
HalfedgeMesh::flipEdge(...)
```

For Your Write-Up: Task 4

- Briefly explain how you implemented the edge flip operation and describe any interesting implementation / debugging tricks you have used.
- Show screenshots of a mesh before and after some edge flips.
- Write about your eventful debugging journey, if you have experienced one.

2.3 Task 5: Edge Split (15 Points + 6 Extra)

In Task 5, you will implement a different local remeshing operation on an edge, called a *split*. Given a pair of triangles (a, b, c) and (c, b, d), a split operation on their shared edge (b, c) inserts a new vertex m at its midpoint and connects the new vertex to each opposing vertex a and d, yielding four triangles, as shown below:



You need to implement this in `HalfedgeMesh::splitEdge(...)` in `student_code.cpp`. This function takes as input an `EdgeIter` to the edge to be split, e.g. edge (b, c) above, and outputs a `VertexIter` to the newly inserted vertex, e.g. vertex m above.

Because an edge split adds new mesh elements (a new vertex, two new triangles, three new edges, etc.), you will have more pointers to keep track of, and may find an edge split a little trickier to implement than an edge flip. We encourage you to again follow our recommendation in Task 4 to make sure that all pointers of all mesh elements point to the right elements in the modified mesh.

Your implementation of `HalfedgeMesh::splitEdge(...)` should:

- Ignore requests to split boundary edges, unless you are trying for extra credit: the function can simply return immediately if either neighbouring face of the edge is a boundary loop. Note that splitting a boundary edge does make sense, but flipping a boundary edge does not. (Can you reason why?)
- Set the position of the new vertex to the midpoint of the original edge. Remember from Task 3 that the `Vertex` class has a member variable `Vector3D position`.
- Perform only a constant amount of work. The cost of splitting a single edge should not be proportional to the mesh size.

- Create only as many new elements as needed. The mesh should not have any elements that are not connected to the rest of the mesh.

Extra Credit (6 points): support edge splits for boundary edges. For this, you will need to carefully read the explanation of boundary loops (the "virtual" boundary faces) in the documentation at the top of `halfEdgeMesh.h`. In short, you split the edge in half, but only split in half the face that is not a boundary.

Sanity Check After loading a model from a `.dae` file, click to select an edge and press S to split it. To verify that your implementation is likely correct, flip some edges that you have split, and split some edges that you have flipped. You can also alternate between flipping and splitting edges many times, in mesh regions that are close together and far apart, and check that the mesh changes correctly. Task 6 will require a working implementation of edge splits.

Complete or use the given function:

```
1 HalfedgeMesh::splitEdge(...)
```

For Your Write-Up: Task 5

- Briefly explain how you implemented the edge split operation and describe any interesting implementation / debugging tricks you have used.
- Show screenshots of a mesh before and after some edge splits.
- Show screenshots of a mesh before and after a combination of both edge splits and edge flips.
- Write about your eventful debugging journey, if you have experienced one.
- If you have implemented support for boundary edges, show screenshots of your implementation properly handling split operations on boundary edges.

2.4 Task 6: Loop Subdivision for Mesh Upsampling (25 Points + 7 + 7 Extra)

Sometimes we may wish to convert a coarse polygon mesh into a higher-resolution one, for better display, more accurate simulation, etc. Such a conversion requires an upsampling algorithm that nicely interpolates or approximates the original mesh data.

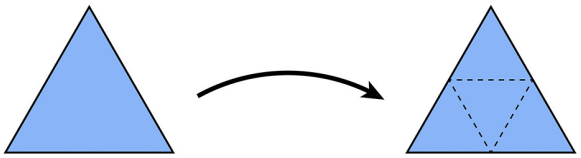
Unfortunately, the techniques we have learned for upsampling 2D images, such as bilinear filtering, do not easily translate to upsampling 3D meshes. Among many reasons, a mesh often has vertices at irregular locations, as opposed to on a regular grid like an image. (Can you think of other reasons?)

In Task 6, you will implement a mesh upsampling method called *Loop subdivision*. In short, Loop subdivision upsamples a mesh by (1) subdividing each of its triangles into four smaller

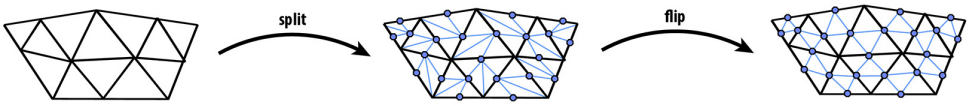
triangles, and (2) updating the vertices of the subdivided mesh based on a weighting scheme.

You need to implement these two steps of Loop subdivision, outlined in more detail below, in `MeshResampler::upsample(...)` in `student_code.cpp`. This function has no output and takes as input a reference to the `HalfedgeMesh` to be subdivided. A single Loop subdivision consists of the following two steps. If we repeatedly apply them, we converge to a smooth approximation of the original mesh.

1. 4-1 subdivision Subdivide each triangle of the mesh into four by connecting the edge midpoints, as shown below. This is called a 4-1 subdivision.



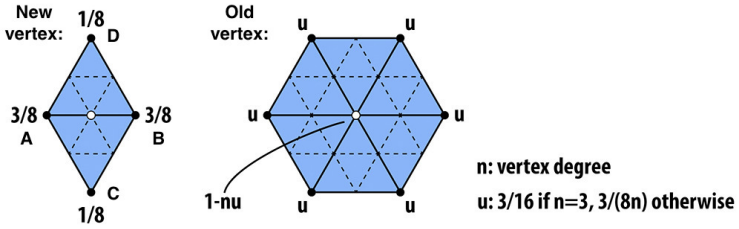
To perform 4-1 subdivision over the entire mesh, you can use the edge flip and edge split operations you have implemented:



(1) Split every existing edge of the mesh, in any order. (2) Flip any *new* edge that connects an old vertex and a new vertex.

Note: after (1), every original edge is represented by 2 edges. You should not flip these edges, because they already lie along the boundaries of the 4-way subdivided triangles. In the figure above, you should only flip the blue edges, which connect an old vertex and a new vertex; you should *not* flip any black new edges.

2. Update vertex positions Update the vertex positions as weighted averages of neighbouring vertex positions. The figure below shows the weights we use (1) to compute the position of a newly added vertex, or (2) to update the position of an existing vertex. The new vertex and the old vertex are depicted as the white circle in their respective meshes.



(1) The position of a new vertex splitting the shared edge (A, B) between a pair of triangles (A, C, B) and (A, B, D) is

```
3/8 * (A + B) + 1/8 * (C + D)
```

(2) The updated position of an old vertex is

```
(1 - n * u) * original_position + u * original_neighbor_position_sum
```

where n is the vertex degree, i.e. the number of edges incident to the vertex; u is the constant shown in the figure ($u = 3/16$ if $n = 3$, and $u = 3/(8n)$ otherwise); `original_position` is the original position of the old vertex; and `original_neighbor_position_sum` is the sum of the original positions of all neighbouring vertices.

Notes While you can implement Loop subdivision exactly as described, we actually recommend that you compute the updated vertex positions *before* performing the 4-1 subdivision over the entire mesh. More specifically, you may want to follow these steps:

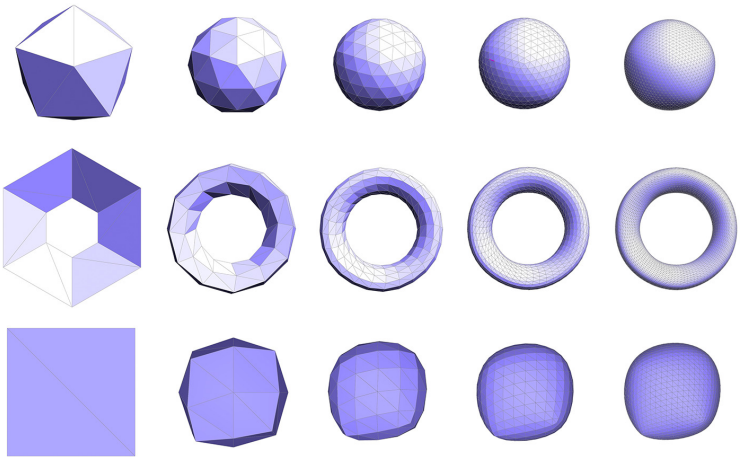
- Step A: compute the positions of both the new and the old vertices using the original mesh. We want to do this before subdivision because traversing a coarse mesh is much easier than traversing a subdivided mesh with more elements.
- Step B: subdivide the original mesh via edge splits and flips as described.
- Step C: update all vertex positions in the subdivided mesh using the values already computed.

If you follow this recommendation, you may use the following member variables of the `Vertex` and `Edge` classes:

- `Vector3D Vertex::newPosition` temporarily stores the updated position of a new or old vertex, computed with the weighted average described above.
- `Vector3D Edge::newPosition` temporarily stores the position of the vertex that will ultimately be inserted at the edge midpoint.
- `bool Vertex::isNew` flags whether a vertex is new, i.e. inserted at an edge midpoint by subdivision, rather than a vertex of the original mesh.
- `bool Edge::isNew` flags whether an edge is new, i.e. added by subdivision, rather than an edge of the original mesh.

Note: while computing the updated vertex positions in Step A, you should not change the value of `Vector3D Vertex::position`, because the weighted averages are based on the vertex positions in the original mesh. (Please take a moment to understand why.) You will, however, need to update `Vertex::position` in Step C.

The examples below illustrate the correct behaviour of the Loop subdivision algorithm:



Extra Credit (7 points): support meshes with boundary. You will first need to make sure that your edge split function handles boundary edges appropriately. You do not need to change your edge flip function. You will also need a different weighted average for boundary vertices; see "A Survey of Subdivision-Based Tools for Surface Modeling" for more information.

Extra Credit (7 points): implement additional subdivision schemes. There exist many alternatives to Loop subdivision. Triangle subdivision schemes include the Butterfly scheme, the modified Butterfly scheme, and $\sqrt{3}$-subdivision. One of the most popular subdivision schemes for quadrilateral meshes is Catmull-Clark. There are also special subdivision schemes, such as polar subdivision for handling meshes with vertices of high degree.

Complete or use the given function:

```
1 MeshResampler::upsample(...)
```

For Your Write-Up: Task 6

- Briefly explain how you implemented Loop subdivision and describe any interesting implementation / debugging tricks you have used.
- Take some notes, and some screenshots, of your observations on how meshes behave after Loop subdivision. What happens to sharp corners and edges? Can you reduce this effect by pre-splitting some edges?
- Load `dae/cube.dae` and perform several iterations of Loop subdivision on the cube. Notice that the cube becomes slightly asymmetric after repeated subdivisions. Can you pre-process the cube with edge flips and splits so that it subdivides symmetrically? Document these effects and explain why they occur. Also explain how your pre-processing helps alleviate them.
- If you have implemented any extra-credit extensions, explain what you did and document how they work with screenshots.

Tips

- Start early. This assignment has multiple parts, and you want to manage your time accordingly. Remember that pesky bugs can waste your time like nothing else, and always keep in mind Hofstadter's Law.
- Tasks 1 and 2 should be relatively straightforward to implement once you understand Bezier curves and surfaces and de Casteljau's algorithm.
- Tasks 3 to 6 will be more difficult to implement correctly right away, since they involve managing pointers, which are exposed to you as iterators. Make sure you test these parts, especially Tasks 4 and 5, together on a few different meshes. While correct behaviour does not imply correct code, incorrect behaviour does imply incorrect code.
- Make sure you allocate enough time to do a good job on the write-up!

Submission

Project Write-Up Guidelines and Instructions Please follow the overall structure described in the deliverables below. The goals of your write-up are for you to (1) think about and articulate what you have built and learned in your own words, and (2) have a write-up of the project to take away from the class. Your write-up should include:

- An overview of the project, including your approach to and implementation of each of the parts, as well as the problems you encountered and how you solved them. Strive for clarity and succinctness.
- For each part, the results described in the corresponding deliverables below, in addition to your explanation. If you failed to generate any results correctly, provide a brief explanation of why.
- A clear indication of any extra-credit items you completed, with a thorough explanation and illustration of each.

The write-up is one of our main methods of evaluating your work, so it is important to spend the time to do it correctly and thoroughly. Plan ahead and allocate time for the write-up well before the deadline.

Project Write-Up Deliverables and Rubric This rubric lists the basic, minimum requirements for your write-up. The content and quality of your write-up are extremely important, and you should make sure to address at least all the points listed below. The extra-credit items are intended for students who want to challenge themselves and explore methods beyond the fundamentals. They are not worth many points, so don't necessarily expect to use extra credit to make up for points lost elsewhere.

Overview Give a high-level overview of what you have implemented in this assignment. Think about what you have built as a whole. Share your thoughts on what interesting things you have learned from completing this assignment.

Task 1

- Briefly explain de Casteljau's algorithm and how you implemented it to evaluate Bezier curves.
- Take a look at the provided `.bzc` files and create your own Bezier curve with 6 control points of your choosing. Use this Bezier curve for the screenshots below.
- Show screenshots of each step / level of the evaluation, from the original control points down to the final evaluated point. Press E to step through. Toggle C to show the completed Bezier curve as well.
- Show a screenshot of a slightly different Bezier curve, obtained by moving the original control points around and modifying the parameter t by mouse scrolling.

Task 2

- Briefly explain how de Casteljau's algorithm extends to Bezier surfaces and how you implemented it to evaluate Bezier surfaces.
- Show a screenshot of `bez/teapot.bez` (not `.dae`) evaluated by your implementation.

Task 3

- Briefly explain how you implemented the area-weighted vertex normals.
- Show screenshots of `dae/teapot.dae` (not `.bez`) comparing the teapot's shading with and without vertex normals. Use Q to toggle between the default flat shading and Phong shading.

Task 4

- Briefly explain how you implemented the edge flip operation and describe any interesting implementation / debugging tricks you have used.
- Show screenshots of a mesh before and after some edge flips.
- Write about your eventful debugging journey, if you have experienced one.

Task 5

- Briefly explain how you implemented the edge split operation and describe any interesting implementation / debugging tricks you have used.

- Show screenshots of a mesh before and after some edge splits.
- Show screenshots of a mesh before and after a combination of both edge splits and edge flips.
- Write about your eventful debugging journey, if you have experienced one.
- If you have implemented support for boundary edges, show screenshots of your implementation properly handling split operations on boundary edges.

Task 6

- Briefly explain how you implemented Loop subdivision and describe any interesting implementation / debugging tricks you have used.
- Take some notes, and some screenshots, of your observations on how meshes behave after Loop subdivision. What happens to sharp corners and edges? Can you reduce this effect by pre-splitting some edges?
- Load `dae/cube.dae` and perform several iterations of Loop subdivision on the cube. Notice that the cube becomes slightly asymmetric after repeated subdivisions. Can you pre-process the cube with edge flips and splits so that it subdivides symmetrically? Document these effects and explain why they occur. Also explain how your pre-processing helps alleviate them.
- If you have implemented any extra-credit extensions, explain what you did and document how they work with screenshots.